

Event System Architecture Summary

Introduction

This document summarizes the collaborative discussion between Manus and ChatGPT regarding the design of an event-driven architecture for a multi-tenant operations engine, integrating CRM functionalities. The core principle established is to treat CRM as a native module within the existing workforce primitives, leveraging a unified event stream as the backbone.

Key Architectural Decisions

1. Unified Strategy: Workforce + CRM = One System

- **Shared Backbone:** The system will maintain a single, unified architecture, avoiding separate products for workforce management and CRM. Core components like tenancy, RBAC, and authentication will be shared.
- **Shared Primitives:** Existing primitives such as tasks, events, inventory, and issues will be extended to support CRM functionalities.
- **New CRM Layer:** A new CRM layer will be introduced for entities like contacts, deals, companies, and activity timelines, built on top of the shared primitives.

2. Event Stream as the Glue

- **Immutable Event Stream:** Events will be modeled as a separate, immutable stream, not as edges in the graph. This ensures a clear audit trail and facilitates cross-module visibility and timeline generation.
- **Event Referencing:** Events will reference existing nodes and edges within the system, rather than replacing them.

3. Canonical Event Schemas

Core Operational Events:

- **task.created**
 - **event_id** : Unique identifier for the event.
 - **timestamp** : Time of event creation.
 - **actor_id** : ID of the user who created the task.

- **task_id** : ID of the newly created task.
- **task_type** : Type of task (e.g., 'cleaning', 'maintenance').
- **description** : Brief description of the task.
- **due_date** : Optional due date for the task.
- **priority** : Task priority (e.g., 'high', 'medium', 'low').
- **assigned_to** : User ID or role ID assigned to the task.
- **related_entities** :
 - **entity_id** : ID of the related entity (e.g., room ID, guest ID).
 - **entity_type** : Type of the related entity (e.g., 'room', 'guest').
 - **role** : Optional role of the entity in relation to the event (e.g., 'primary', 'secondary').
- **booking.created**
 - **event_id** : Unique identifier for the event.
 - **timestamp** : Time of event creation.
 - **actor_id** : ID of the user or system that created the booking.
 - **booking_id** : ID of the newly created booking.
 - **guest_id** : ID of the guest associated with the booking.
 - **room_id** : ID of the room booked.
 - **check_in_date** : Check-in date.
 - **check_out_date** : Check-out date.
 - **total_price** : Total price of the booking.
 - **status** : Initial status of the booking (e.g., 'confirmed', 'pending').
 - **related_entities** :
 - **entity_id** : ID of the related entity (e.g., room ID, guest ID).
 - **entity_type** : Type of the related entity (e.g., 'room', 'guest').
 - **role** : Optional role of the entity in relation to the event (e.g., 'primary', 'secondary').
- **task.updated**
 - **event_id** : Unique identifier for the event.
 - **timestamp** : Time of event update.
 - **actor_id** : ID of the user who updated the task.

- `task_id` : ID of the updated task.
- `changes` :
 - `field` : Name of the field that was changed (e.g., 'status', 'assigned_to').
 - `old_value` : Previous value of the field.
 - `new_value` : New value of the field.
- `related_entities` :
 - `entity_id` : ID of the related entity.
 - `entity_type` : Type of the related entity.
 - `role` : Optional role of the entity.
- **`task.completed`**
 - `event_id` : Unique identifier for the event.
 - `timestamp` : Time of task completion.
 - `actor_id` : ID of the user who completed the task.
 - `task_id` : ID of the completed task.
 - `duration` : Optional time taken to complete the task.
 - `notes` : Optional notes on task completion.
 - `related_entities` :
 - `entity_id` : ID of the related entity.
 - `entity_type` : Type of the related entity.
 - `role` : Optional role of the entity.

CRM Events:

- **`deal.created`**
 - `event_id` : Unique identifier for the event.
 - `timestamp` : Time of deal creation.
 - `actor_id` : ID of the user who created the deal.
 - `deal_id` : ID of the newly created deal.
 - `deal_name` : Name of the deal.
 - `amount` : Monetary value of the deal.
 - `stage` : Current stage of the deal (e.g., 'prospecting', 'qualification').
 - `contact_id` : ID of the primary contact for the deal.

- `organization_id` : ID of the organization associated with the deal.
- `related_entities` :
 - `entity_id` : ID of the related entity.
 - `entity_type` : Type of the related entity.
 - `role` : Optional role of the entity.
- **`deal.stage_changed`**
 - `event_id` : Unique identifier for the event.
 - `timestamp` : Time of stage change.
 - `actor_id` : ID of the user who changed the deal stage.
 - `deal_id` : ID of the deal.
 - `old_stage` : Previous stage of the deal.
 - `new_stage` : New stage of the deal.
 - `notes` : Optional notes on the stage change.
 - `related_entities` :
 - `entity_id` : ID of the related entity.
 - `entity_type` : Type of the related entity.
 - `role` : Optional role of the entity.
- **`contact.created`**
 - `event_id` : Unique identifier for the event.
 - `timestamp` : Time of contact creation.
 - `actor_id` : ID of the user who created the contact.
 - `contact_id` : ID of the newly created contact.
 - `first_name` : First name of the contact.
 - `last_name` : Last name of the contact.
 - `email` : Email address of the contact.
 - `phone` : Phone number of the contact.
 - `organization_id` : Optional ID of the organization the contact belongs to.
 - `related_entities` :
 - `entity_id` : ID of the related entity.
 - `entity_type` : Type of the related entity.
 - `role` : Optional role of the entity.

4. Front-end API for Event Consumption

- **Endpoint:** GET /api/v1/timeline
- **Parameters:**
 - `entity_id` (required): The ID of the entity for which to retrieve the timeline (e.g., `room_id` , `guest_id` , `deal_id` , `contact_id`).
 - `entity_type` (required): The type of the entity (e.g., `room` , `guest` , `deal` , `contact`).
 - `start_date` (optional): Filter events from this date onwards (ISO 8601 format).
 - `end_date` (optional): Filter events up to this date (ISO 8601 format).
 - `event_types` (optional): Comma-separated list of event types to include (e.g., `task.created,deal.stage_changed`).
 - `limit` (optional): Maximum number of events to return (default: 50).
 - `offset` (optional): Number of events to skip for pagination (default: 0).
- **Sample Response:**

JSON

```
{
  "total_events": 120,
  "limit": 50,
  "offset": 0,
  "events": [
    {
      "event_id": "evt_abc123",
      "event_type": "task.created",
      "timestamp": "2026-03-19T10:00:00Z",
      "actor_id": "user_123",
      "payload": {
        "task_id": "task_456",
        "task_type": "cleaning",
        "description": "Clean Room 101",
        "priority": "high",
        "assigned_to": "user_789"
      },
      "related_entities": [
        {
          "entity_id": "room_101",
          "entity_type": "room",
          "role": "primary"
        }
      ]
    },
    {

```

```
"event_id": "evt_def456",
"event_type": "deal.stage_changed",
"timestamp": "2026-03-19T11:30:00Z",
"actor_id": "user_101",
"payload": {
  "deal_id": "deal_789",
  "old_stage": "prospecting",
  "new_stage": "qualification"
},
"related_entities": [
  {
    "entity_id": "deal_789",
    "entity_type": "deal",
    "role": "primary"
  },
  {
    "entity_id": "contact_222",
    "entity_type": "contact",
    "role": "associated"
  }
]
}
]
```

Conclusion

The collaborative effort has resulted in a well-defined event-driven architecture that unifies workforce and CRM functionalities. The detailed event schemas and the proposed front-end API provide a clear roadmap for the next phase of implementation. The next steps will involve building out the timeline component and integrating these event streams into the platform.